

ЗВІТ

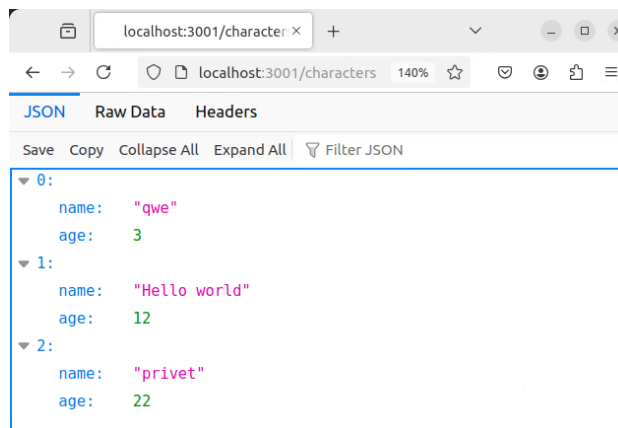
за результатами тестування

Налаштування середовища

Для проведення навантажувального тестування за допомогою JMeter, я використав контейнер з додатком QA Pro REST App, доступний на DockerHub за посиланням [oleksandrgolubishko/qa_pro_rest_app](https://hub.docker.com/oleksandrgolubishko/qa_pro_rest_app). Контейнер був запущений за допомогою команди

```
docker run -p 3001:3001 oleksandrgolubishko/qa_pro_rest_app
```

Після запуску контейнера, доступ до додатку можна було отримати за адресою <http://localhost:3001/characters>, що підтверджує правильність налаштувань. Також, я ознайомився з документацією API додатку, що дозволило створити необхідні запити для тестування.



Створення Тест-Плану

Я розробив тест-план у JMeter для тестування REST API, який включає наступні кроки:

API запити (було створено кілька типів запитів (GET, POST, PUT та DELETE) для різних кінцевих точок API згідно з документацією - [API documentation](#)):

get_characters

GET <http://10.0.104.80:3001/characters>

Status: 200

Response Headers:

X-Powered-By: Express

Content-Type: application/json; charset=utf-8

Content-Length: 83

ETag: W/"53-V8BaVX6DsYMo3t9AbB5N8Bz1MN0"

Date: Thu, 16 Jan 2025 17:29:20 GMT

Connection: keep-alive

Keep-Alive: timeout=5

Response Body

```
[{"name":"qwe","age":3}, {"name":"Hello world","age":12}, {"name":"privet","age":22}]
```

get_character/id

GET http://10.0.104.80:3001/character/1

Status 200

Response Headers:

X-Powered-By: Express

Content-Type: application/json; charset=utf-8

Content-Length: 22

ETag: W/"16-6td2MKJUfoOmCmb1rhT8SJ4FakQ"

Date: Thu, 16 Jan 2025 17:38:50 GMT

Connection: keep-alive

Keep-Alive: timeout=5

Response Body

```
{"name":"qwe","age":3}
```

post_character

POST http://10.0.104.80:3001/character/4

Status 200

Request Body

```
{
  "name": "Example",
  "age": "111"
}
```

Response Headers

X-Powered-By: Express

Content-Type: application/json; charset=utf-8

Content-Length: 30

ETag: W/"1e-jepjN1vVz6kRai69HbukuTqsiLk"

Date: Thu, 16 Jan 2025 17:44:24 GMT

Connection: keep-alive

Keep-Alive: timeout=5

Response Body

```
{"name":"Example","age":"111"}
```

put_character

PUT http://10.0.104.80:3001/character/4

Status 200

Request Body

```
{
  "name": "${__RandomString(8,abcdefghijklmnopqrstuvwxyz)}",
  "age": "${__Random(16,80)}"
}
```

Response Headers

```
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 28
ETag: W/"1c-ESXJvKvLQ3+1jvVb2OvZbaAk2R0"
Date: Thu, 16 Jan 2025 17:52:46 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Response Body

```
{"name":"manjmhrk","age":65}
```

В Request Body використано функцій JMeter, які використовуються для генерації випадкових значень.

delete_character

DELETE http://10.0.104.80:3001/character/4

Status 200

Response Headers

```
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 15
ETag: W/"f-NQVRIQfKHCoInEbhaLgECMonhCE"
Date: Thu, 16 Jan 2025 18:00:15 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Response Body

```
{"status":true}
```

При запиті в id використовував вбудовану функцію JMeter для генерації випадкових значень "\${__Random(1,100)}"

Thread Group (для кожного рівня навантаження визначено параметри):

- Thread users - кількість одночасних користувачів;
- Ramp up - час, за який повинно бути розгорнуто задану кількість користувачів;
- Loop count - кількість повторів для кожного користувача.
- Duration - для кожного запиту додано умови, які перевіряють, чи час виконання не перевищує задану величину.

Configuration 1			Configuration 2			
Thread users 100	Ramp up 1	Loop count 1	Thread users 1000	Ramp up 2	Loop count Infinity	Duration 30
Thread users 1000	Ramp up 1	Loop count 2	Thread users 5000	Ramp up 3	Loop count Infinity	Duration 60
Thread users 5000	Ramp up 1	Loop count 4	Thread users 10000	Ramp up 5	Loop count Infinity	Duration 180

HTTP Request Defaults (налаштування базового URL для API):

URL було винесено в змінну

Stress Testing

Target Concurrency: 10 - **кількість одночасних користувачів (потоків), які JMeter створює для тестування, 10 одночасних потоків**

Ramp Up Time:10 - **поступово збільшуватиме кількість потоків від 0 до 10**

Ramp Up Step Count:10 - Up Time (10 секунд) ділиться на 10 етапи:

$10 \text{ потоків} \div 10 \text{ етапи} = 1 \text{ потоки за етап.}$

Hold Target Rate Time (min):10 - це час, протягом якого JMeter **утримує максимальне навантаження з 10 потоків** після завершення Ramp Up Time.

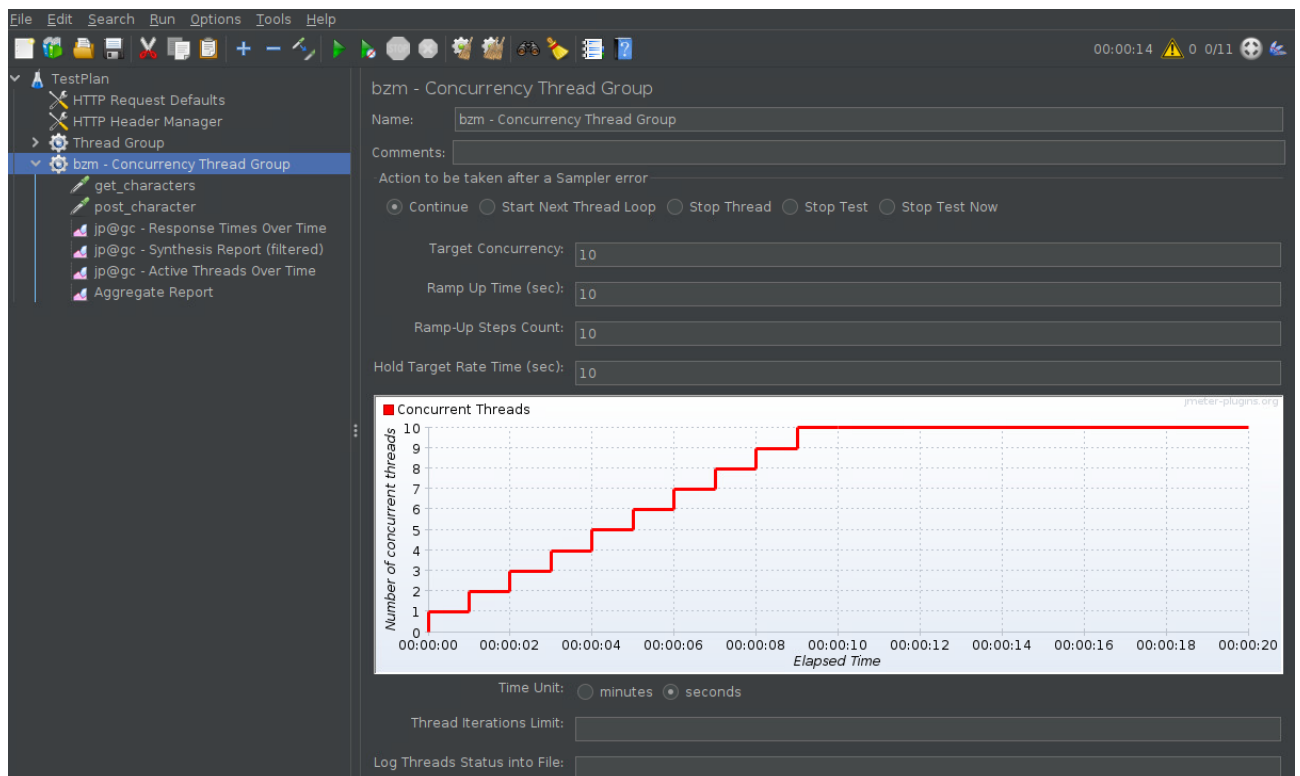
1-й потік: на 1-й секунді.

2-й потік: на 2-й секунді.

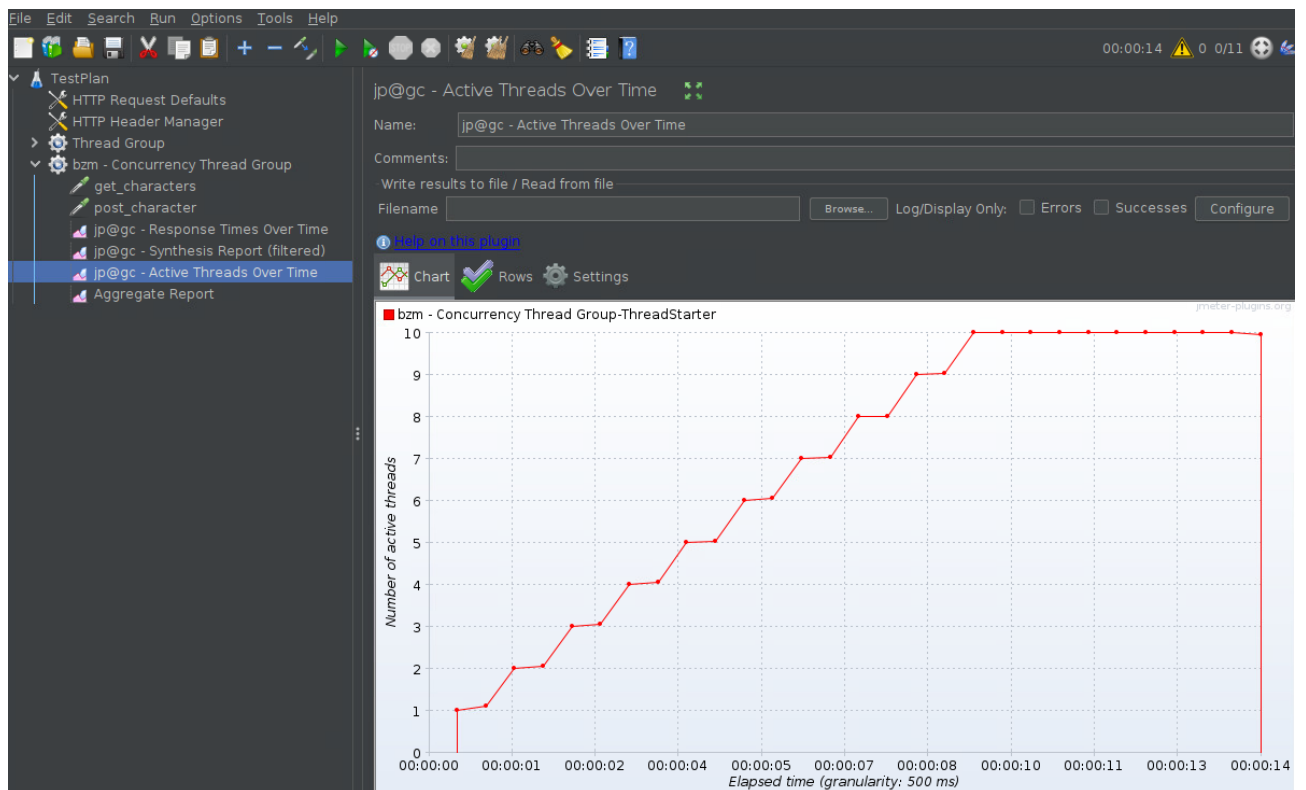
3-й потік: на 3-й секунді.

І так далі, доки не буде створено всі 10 потоків до кінця 10 секунд.

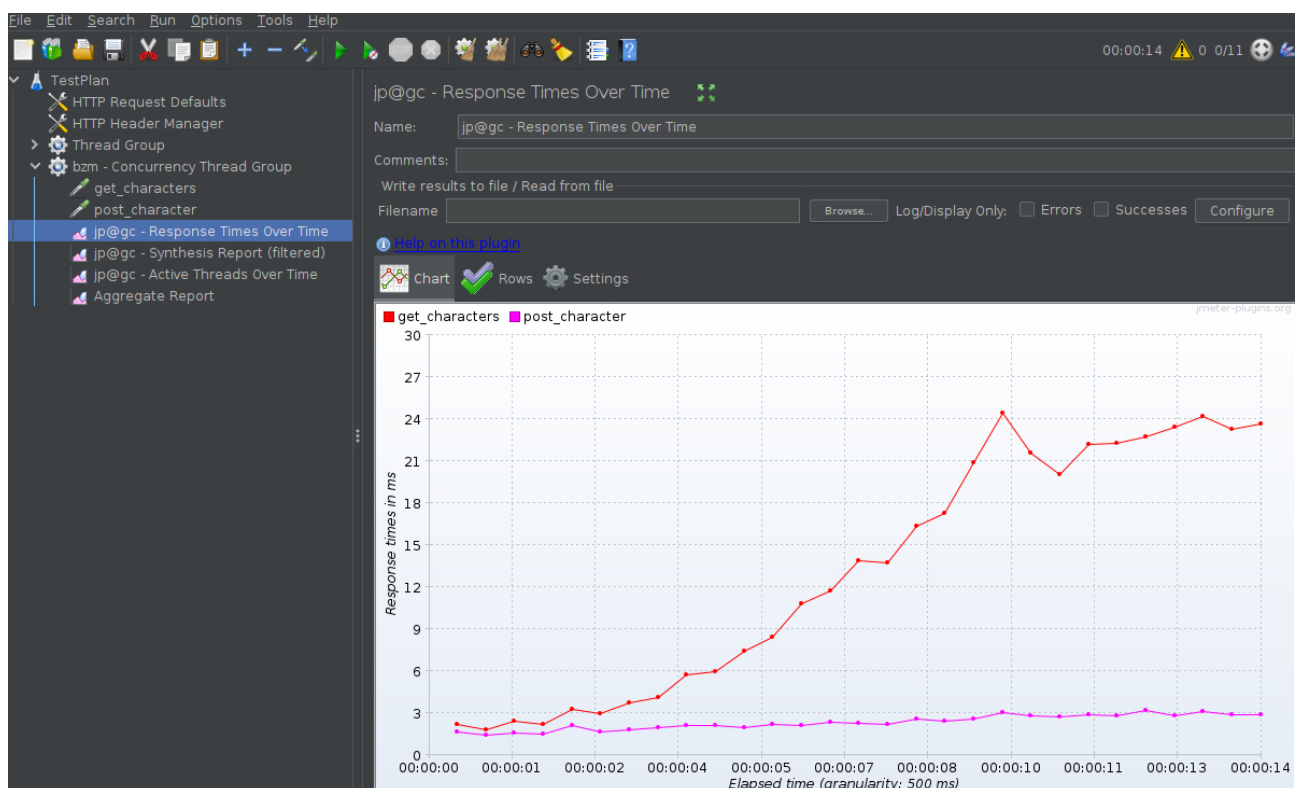
Отже, встановлюємо вказані налаштування.



Запускаємо тестування і бачимо в Active Threads Over Time, що кількість Threads збігається з тим що має бути.



Також у Response Time Over Time спостерігаємо що кожен запит здійснюється окремо, відповідь в мс



Отже, проаналізуємо та зробимо висновки з Aggregate Report:

TestPlan.jmx (/home/user/Downloads/apache-jmeter-5.6.3/bin/TestPlan.jmx) - Apache JMeter (5.6.3)

File Edit Search Run Options Tools Help

00:00:14 0 0/11

TestPlan

- HTTP Request
- HTTP Header
- Thread Group
- get_characters
 - post_characters
 - jp@gc - Re
 - jp@gc - Sy
 - jp@gc - Ac
 - Aggregate

Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes Configure

Label ↓	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Received...	Sent KB/...
post_cha...	6256	2	2	3	4	7	0	20	0.00%	434.9/sec	112.52	93.83
get_char...	6265	12	11	23	27	31	1	52	0.05%	434.2/sec	41146.73	67.39
TOTAL	12521	7	3	21	23	30	0	52	0.02%	867.8/sec	41258.90	160.94

☐ Include group name in label? Save Table Data ☒ Save Table Header

Кількість зразків (Samples):

- post_characters: 6256 запитів.
- get_characters: 6256 запитів.

Загалом: 12,521 запит.

Час відповіді (Response Time):

- **Середній час (Average):**
 - post_characters: 2 мс.
 - get_characters: 12 мс.

Загалом: 7 мс.

- **Медіана (Median):**
 - post_characters: 2 мс.
 - get_characters: 11 мс.

Загалом: 3 мс.

- **90-й, 95-й та 99-й процентиль:**
 - post_characters: відповідно 3, 4, 7 мс.
 - get_characters: відповідно 23, 27, 31 мс.

Загалом: відповідно 21, 23, 30 мс.

- Час відповіді для post_characters значно менший, ніж для get_characters.

Максимальний час відповіді (Maximum):

- post_characters: 20 мс.
- get_characters: 52 мс.

Загалом: 52 мс.

Мінімальний час відповіді (Minimum):

- post_characters: 0 мс.
- get_characters: 1 мс.

Загалом: 0 мс.

Помилки (Error %)

- post_characters: 0.00% (немає помилок).
- get_characters: 0.05% (кілька помилок).

Загальний рівень помилок: 0.02%. Це свідчить про стабільність системи.

Пропускна здатність (Throughput):

- post_characters: 434.9 запитів/сек.
- get_characters: 434.2 запитів/сек.

Загалом: 867.8 запитів/сек.

- Пропускна здатність збалансована між двома типами запитів.

Передані дані:

- **Отримано (Received KB/sec):**

- post_characters: 112.52 KB/сек.
- get_characters: 41146.73 KB/сек.

Загалом: 41258.90 KB/сек.

- **Надіслано (Sent KB/sec):**

- post_characters: 93.83 KB/сек.
- get_characters: 67.39 KB/сек.

Загалом: 160.94 KB/сек.

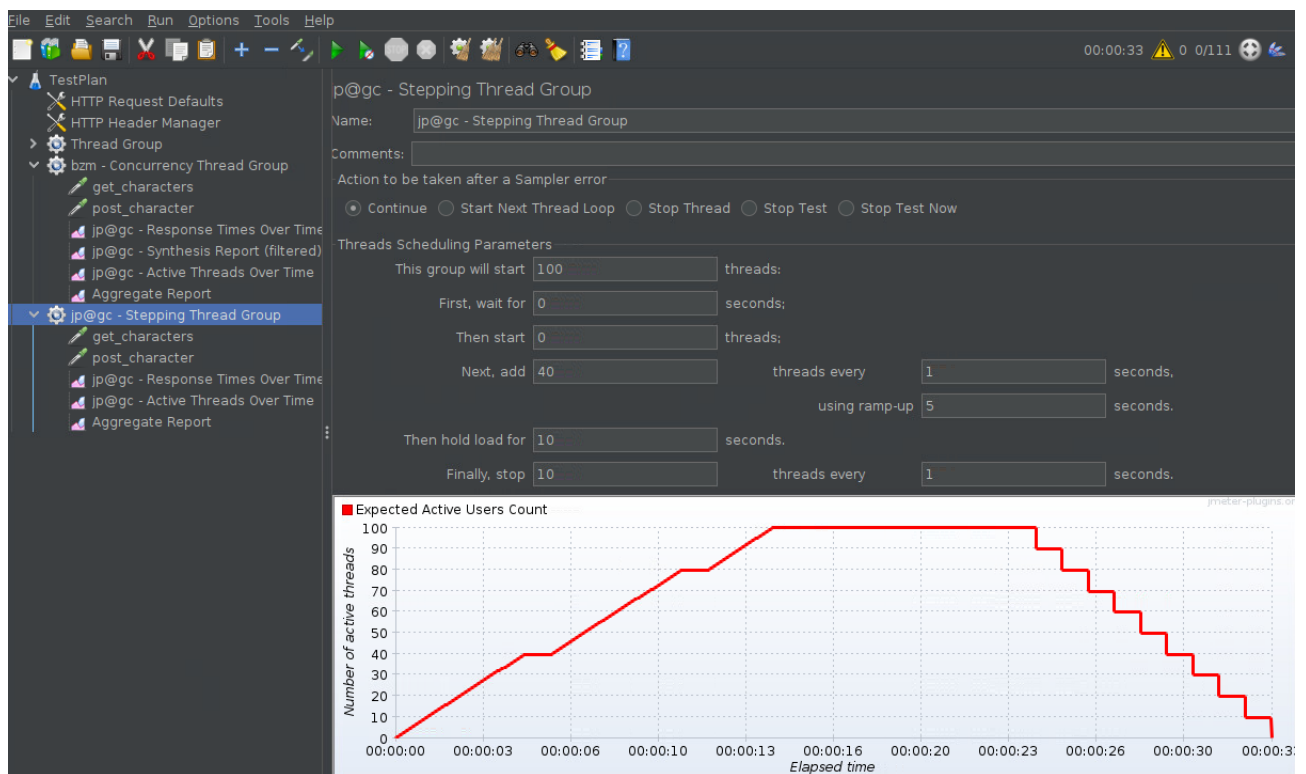
Висновки:

- **post_characters** запити обробляються швидше з меншим середнім та максимальним часом відповіді.

- **get_characters** мають трохи довший час відповіді та невеликий відсоток помилок (0.05%).
- Загальна продуктивність додатку є задовільною, із високою пропускнуою здатністю та низьким рівнем помилок.
- Потенційно можна оптимізувати **get_characters**, щоб зменшити максимальний час відповіді.

Load Testing

Показано налаштування навантажувального тестування за допомогою Stepping Thread Group у JMeter. Мета такого налаштування – поступово збільшувати навантаження на систему, підтримувати його на певному рівні та зменшувати для аналізу поведінки системи під час різних стадій тестування.



Основні параметри налаштувань:

У тесті встановлено, що максимальна кількість одночасних потоків (користувачів) дорівнює **100**;

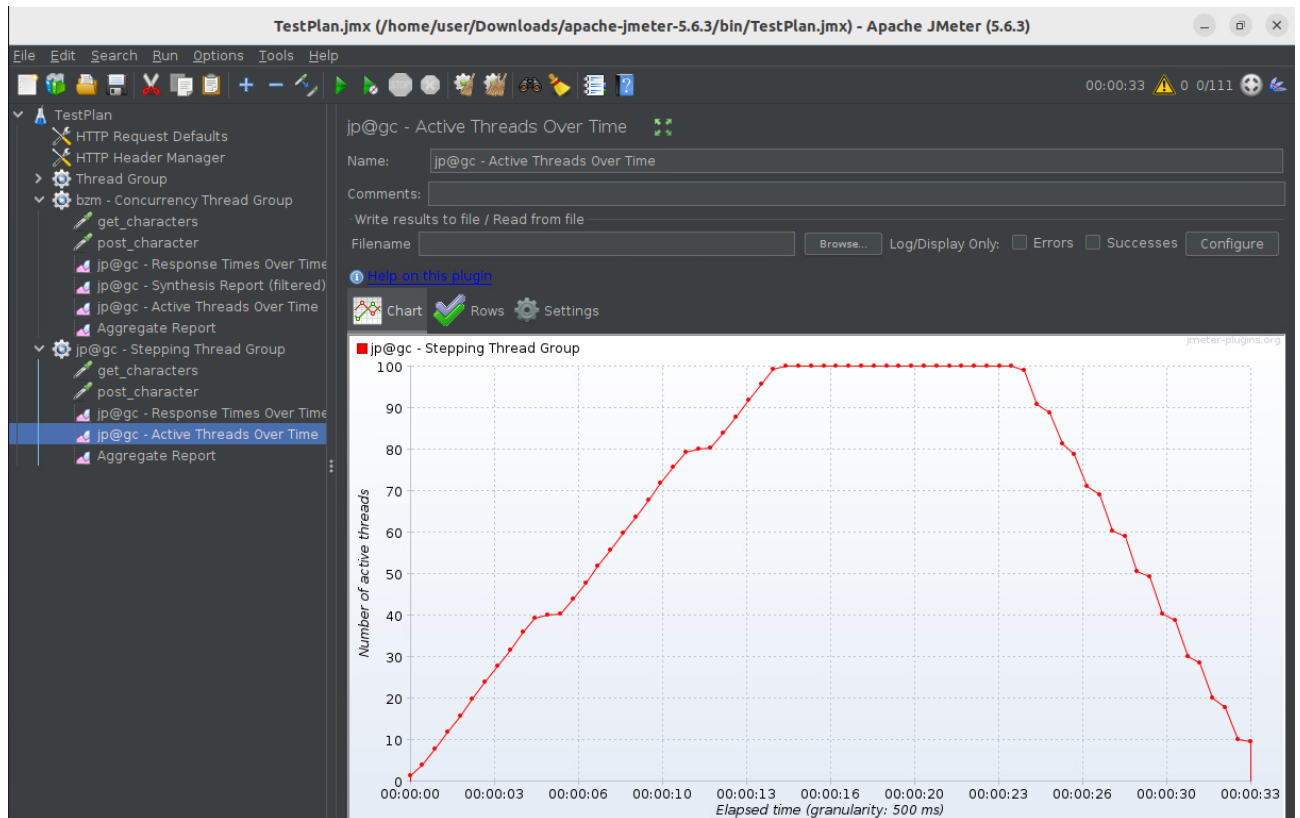
Тест починається **без затримки**, тобто одразу після запуску;

Потоки додаються поступово: кожну **1 секунду** додається **40 потоків**;

Для цього використовується механізм ramp-up, який плавно розподіляє додавання нових потоків протягом **5 секунд**, щоб уникнути різкого стрибка навантаження;

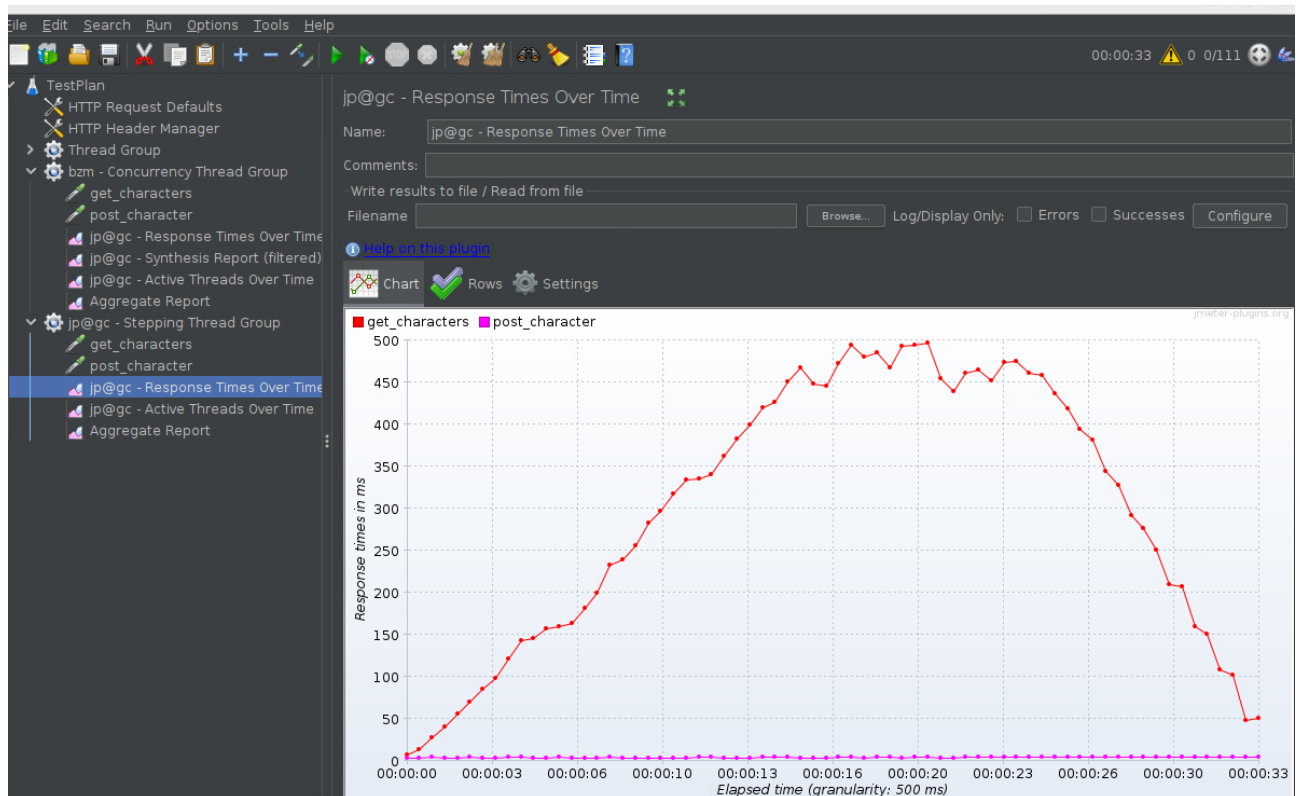
Після досягнення максимальної кількості потоків (100), це навантаження підтримується протягом **10 секунд**;

Потоки поступово зменшуються: кожну секунду закривається **1 потік**, доки їх кількість не зменшиться до нуля;



Графік у нижній частині зображення ілюструє зміну кількості активних потоків (Expected Active Users Count) протягом часу:

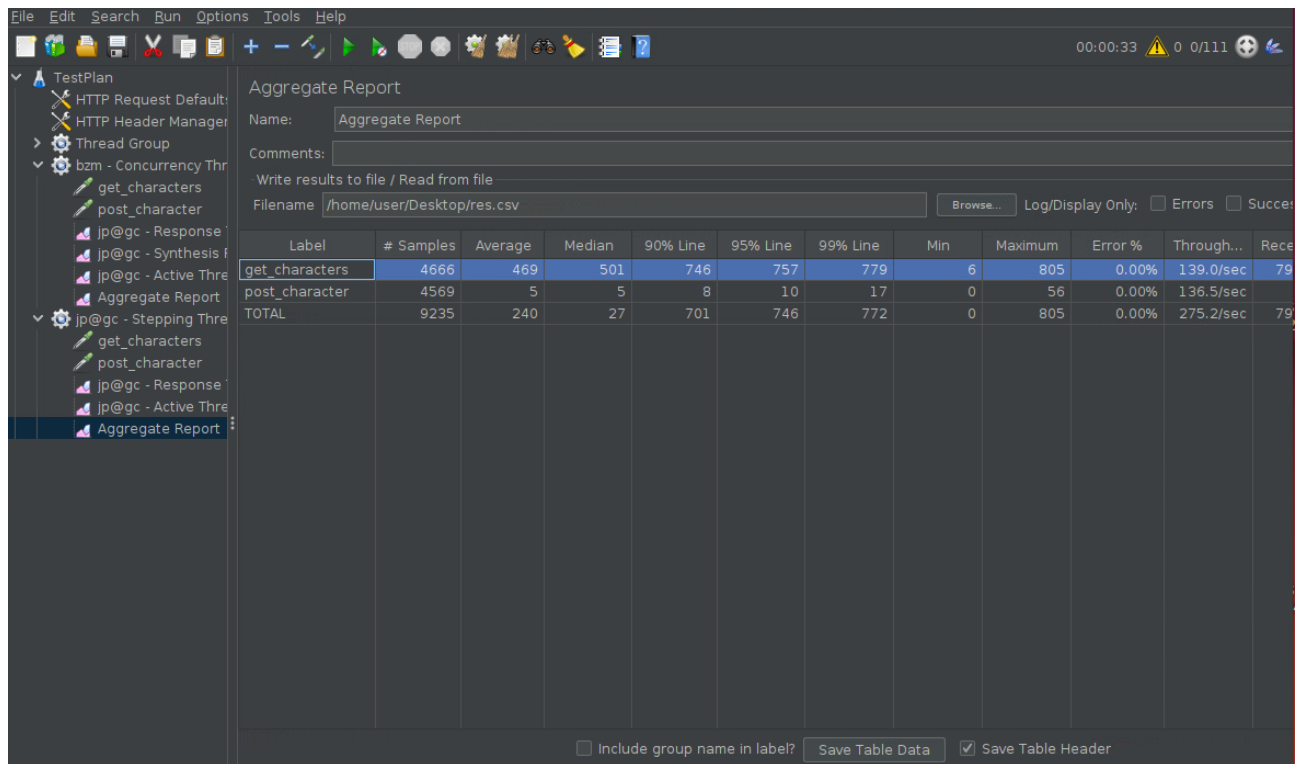
- На початку графік показує поступове збільшення кількості потоків до **100**.
- Після цього кількість потоків утримується на рівні **100** протягом **10 секунд**.
- Наприкінці графік демонструє плавне зменшення навантаження до нуля.



Мета тесту:

- Перевірити, як система реагує на поступове збільшення навантаження.
- Оцінити її продуктивність на піковому рівні (100 одночасних користувачів).
- Аналізувати відновлення системи під час поступового зменшення навантаження.

Такі тести дозволяють виявити, чи здатна система витримувати високі навантаження, чи виникають помилки, та як швидко вона відновлює працездатність.



The screenshot shows the JMeter Aggregate Report window. The left sidebar lists the test plan structure, including 'TestPlan', 'HTTP Request Defaults', 'HTTP Header Manager', 'Thread Group', 'bzm - Concurrency Thread Group', and 'jp@gc - Stepping Thread Group'. The main area displays the 'Aggregate Report' for the 'Aggregate Report' test. The report includes a table with the following data:

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received
get_characters	4666	469	501	746	757	779	6	805	0.00%	139.0/sec	79
post_character	4569	5	5	8	10	17	0	56	0.00%	136.5/sec	79
TOTAL	9235	240	27	701	746	772	0	805	0.00%	275.2/sec	79

At the bottom of the report, there are checkboxes for 'Include group name in label?' (unchecked), 'Save Table Data' (checked), and 'Save Table Header' (checked).

Висновки

Запит `get_characters` має значно вищі показники часу відповіді порівняно з `post_character`, що може свідчити про високе навантаження на сервер або неефективність у обробці цього запиту.

Запит `post_character` має відмінні результати з низькими середніми та максимальними часами відповіді, що свідчить про стабільну та швидку роботу цього запиту.

Є коливання в часах для `get_characters`, де максимальний час відповіді значно більший за мінімальний. Це може бути ознакою проблем з обробкою запиту або змінами в мережі чи сервері.

Потрібно поліпшити продуктивність та звернути увагу на запит `get_characters` та оптимізувати його, оскільки цей запит має більш значні часи відповіді.

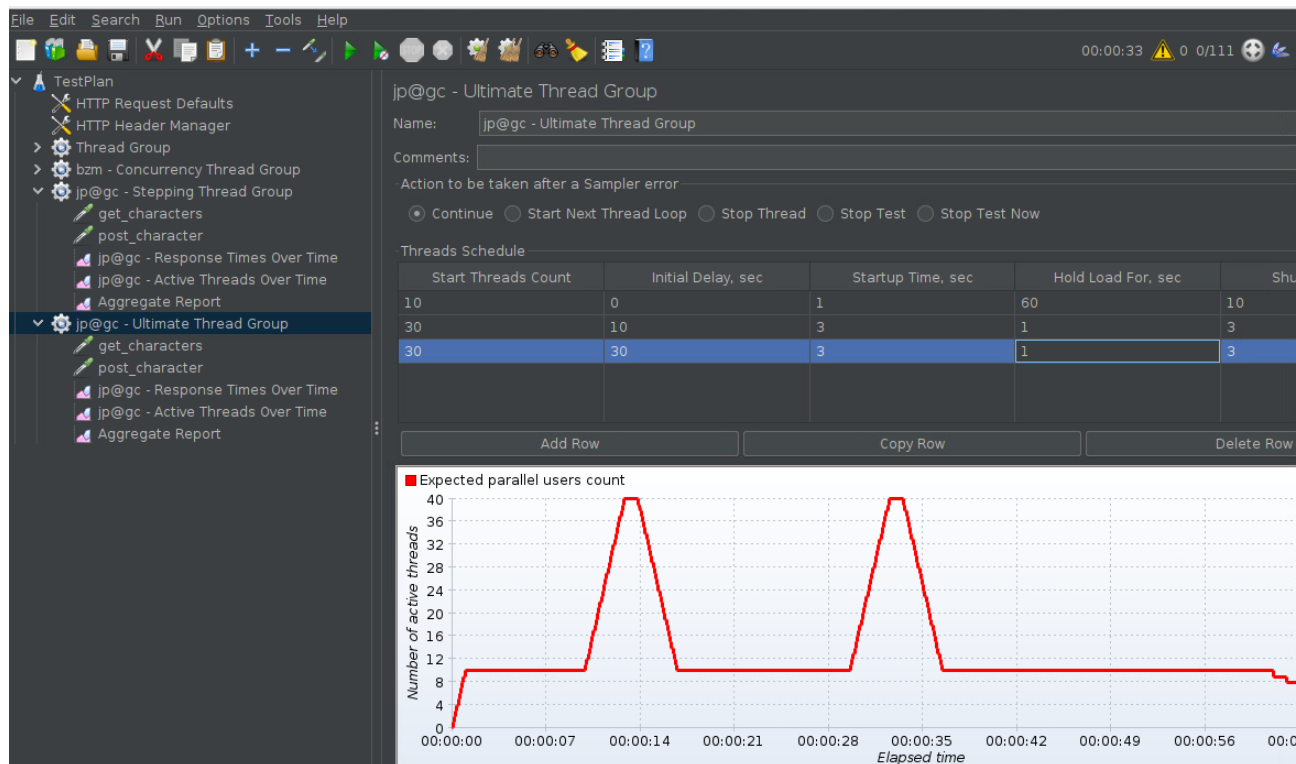
Аналогічним чином можна реалізувати і Endurance testing

Peak testing

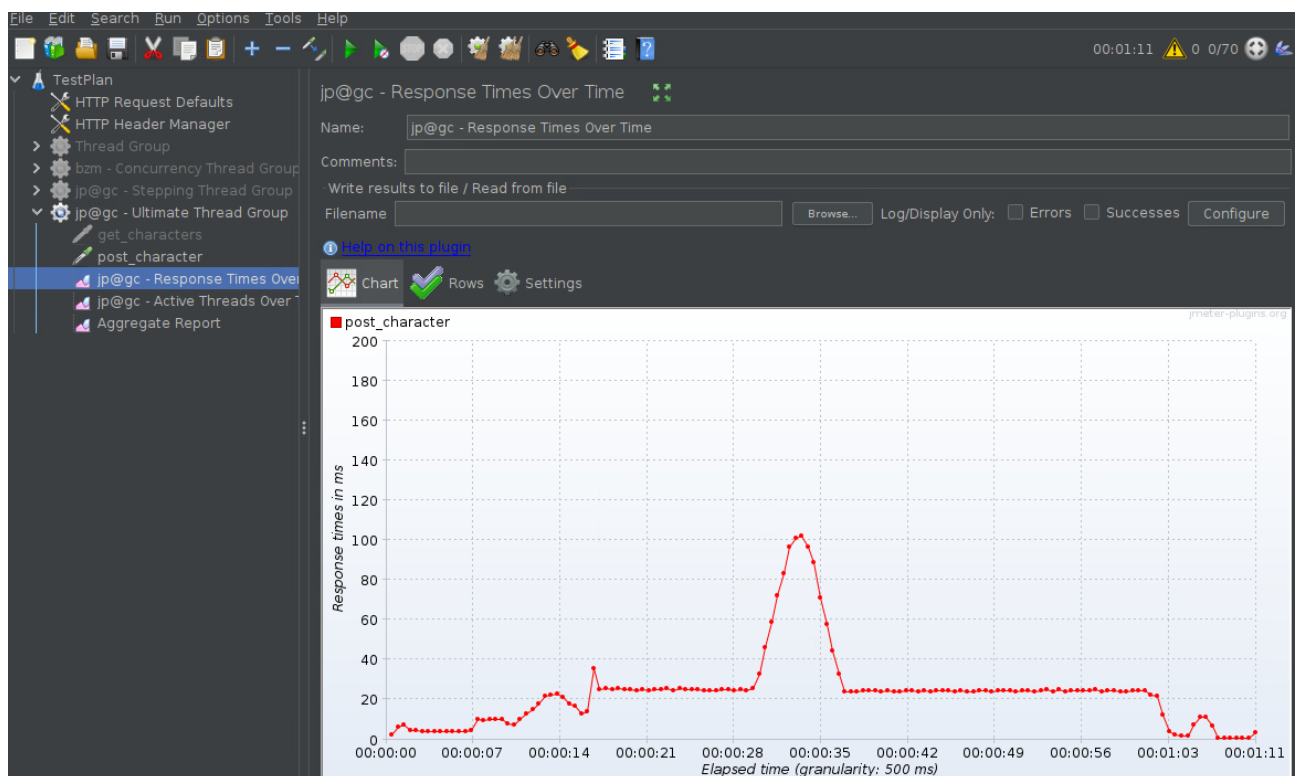
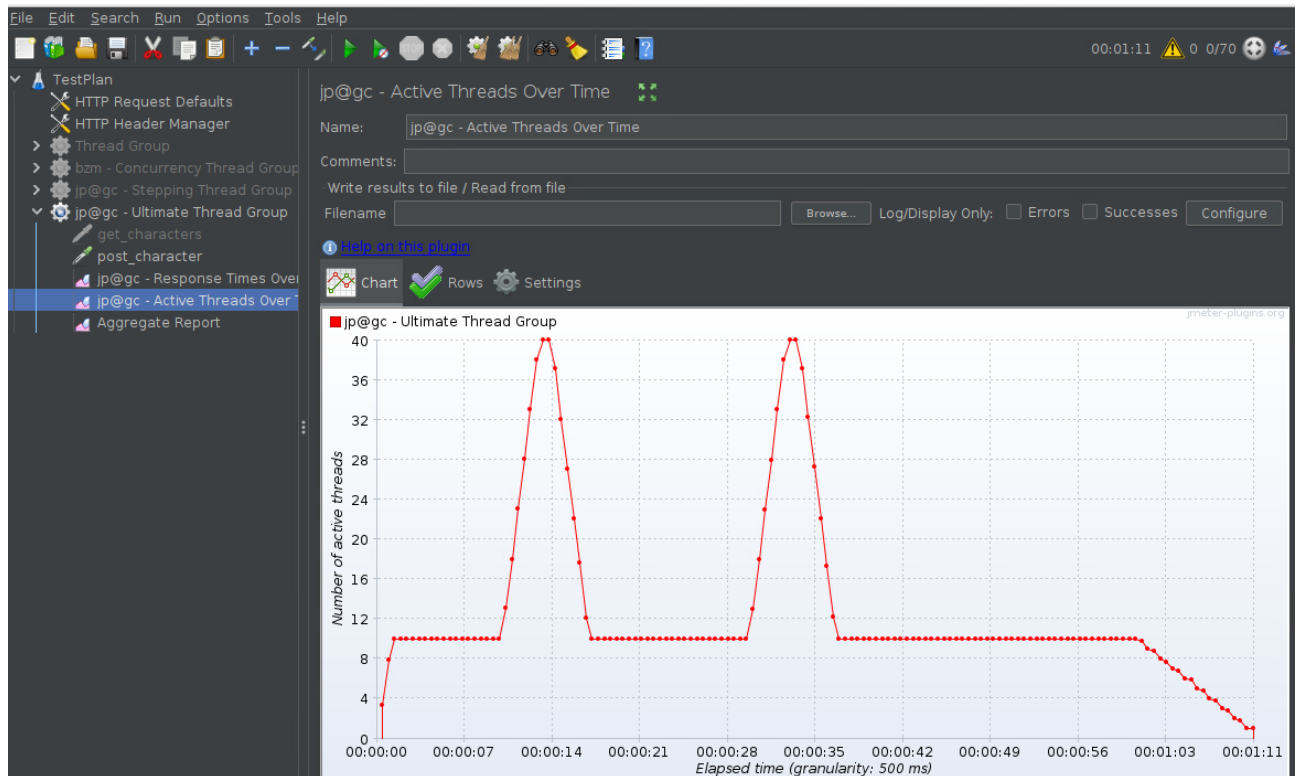
Для проведення Peak testing в Ultimate Thread Group в JMeter, ви налаштовуєте параметри таким чином, щоб змодельовати стресове навантаження на систему, максимізуючи навантаження протягом короткого періоду часу для виявлення можливих пікових проблем.

Використовуємо типові параметри для Peak testing: кількість потоків, час перед початком навантаження, час запуску, час тривалості, час припинення навантаження.

Ось типовий набір параметрів для Peak testing в Ultimate Thread Group, на скріншоті.



Ця конфігурація є хорошим варіантом для **стрес-тестів**: вона створює різке навантаження з високою кількістю потоків, але на дуже короткий час. Це дозволяє перевірити, як система витримує різке навантаження і відновлюється після нього, також перевіряє систему на короткочасне навантаження, але додає затримку перед запуском, що може бути корисно для вивчення поведінки системи в реальних умовах, коли навантаження не починається миттєво.



Розглянемо отримані результати Aggregate Report для вашого тестування в JMeter, де наведено основні показники, такі як кількість зразків, середній час, медіана, 90% та 95% процентилі, максимальний і мінімальний час, а також процент помилок.

The screenshot shows the JMeter 'Aggregate Report' window. The left sidebar lists the test plan structure, including 'TestPlan', 'HTTP Request Defaults', 'HTTP Header Manager', 'Thread Group', 'bzm - Concurrency Thread Group', 'jp@gc - Stepping Thread Group', 'jp@gc - Ultimate Thread Group', 'get_characters', 'post_character', 'jp@gc - Response Times Over', 'jp@gc - Active Threads Over', and 'Aggregate Report'. The main area displays a table of statistics for the 'post_ch...' test element.

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throug...	Receiv...
post_ch...	62130	14	8	31	38	93	0	208	31.61%	876.3/s...	822.4
TOTAL	62130	14	8	31	38	93	0	208	31.61%	876.3/s...	822.4

At the bottom of the window, there are checkboxes for 'Include group name in label?' (unchecked), 'Save Table Data' (button), and 'Save Table Header' (checked).

Ось що означають ці результати:

Samples (Кількість зразків): 62130

○ Це кількість запитів, які були оброблені під час тесту. 62130 зразків свідчать про великий обсяг тестування, що дозволяє отримати детальну картину про ефективність вашої системи під навантаженням.

Average (Середнє значення): 14 мс

○ Середній час обробки запиту становить 14 мс, що є досить хорошим результатом для більшості веб-систем, однак необхідно враховувати й інші фактори, щоб оцінити стабільність та витривалість системи під навантаженням.

Median (Медіана): 8 мс

○ Медіанний час — це середнє значення, яке показує, що половина запитів оброблялася за час менший або рівний 8 мс. Це свідчить про те, що більшість запитів оброблялися швидко.

90% (90-й перцентиль): 31 мс

○ 90% запитів мали час обробки до 31 мс. Це означає, що більшість запитів виконувалася в межах 31 мс, що також є дуже хорошим показником.

95% (95-й перцентиль): 38 мс

○ 95% запитів оброблялося за 38 мс або менше. Це також хороший результат, що свідчить про стабільну роботу системи, оскільки більшість запитів займають відносно мало часу.

99% (99-й перцентиль): 93 мс

○ 99% запитів оброблялося за 93 мс або менше. Це вже значно більший час, ніж для 90% запитів, але все ще знаходиться в межах прийнятного для більшості веб-додатків.

MIN (Мінімальний час): 0 мс

○ Мінімальний час 0 мс означає, що деякі запити були оброблені миттєво. Це може бути результатом дуже легких запитів або кешування.

MAX (Максимальний час): 208 мс

- Максимальний час обробки запиту 208 мс. Це не дуже велика затримка, однак значення в межах 200 мс можуть свідчити про збої або перевантаження під час певних запитів. Це потребує додаткового аналізу, щоб зрозуміти, чому час обробки деяких запитів був таким високим.

Errors (Помилки): 31,61%

- 31,61% — це дуже високий рівень помилок. Майже третина запитів не вдалося виконати успішно. Це є серйозною проблемою і свідчить про те, що ваша система не витримує навантаження, а також вказує на потенційні проблеми з конфігурацією або інфраструктурою.

- Помилки можуть виникати через різні фактори, такі як:
 - Перевантаження серверів.
 - Проблеми з мережею.
 - Помилки у налаштуванні JMeter або конфігурації серверів.

Висновки:

Загальна продуктивність:

- У цілому, середні значення часу (середнє, медіана, 90%, 95%) досить низькі, що свідчить про хорошу швидкість обробки запитів за умови нормального навантаження.

- Однак час на 99-му процентилі (93 мс) і максимальний час (208 мс) показують наявність певних затримок, які потребують додаткового аналізу.

Проблеми з помилками:

- 31,61% помилок — це дуже високий рівень, і він є основною проблемою. Це вказує на те, що ваша система не здатна обробляти таке навантаження без помилок. Вам слід детально аналізувати, чому виникають ці помилки:

- Перевірте серверні логи, щоб з'ясувати причину помилок.
- Можливо, проблема в ресурсах (наприклад, CPU, пам'ять) або в налаштуваннях веб-сервера.
- Аналізуйте типи помилок, щоб зрозуміти, чи є певні запити, які викликають більше помилок.

Рекомендації:

- Потрібно оптимізувати систему для зменшення кількості помилок. Можливо, потрібно налаштувати сервер або додатково масштабувати інфраструктуру для обробки більшої кількості запитів.

- Перевірте налаштування кешування та обробки запитів на сервері, що може допомогти покращити максимальний час обробки.

- Проводьте додаткові тести для аналізу, чи є певні моменти навантаження, коли система починає збільшувати кількість помилок.

В цілому, вам слід зосередитись на вирішенні проблем з високим рівнем помилок і оптимізації системи для зниження максимального часу відповіді.

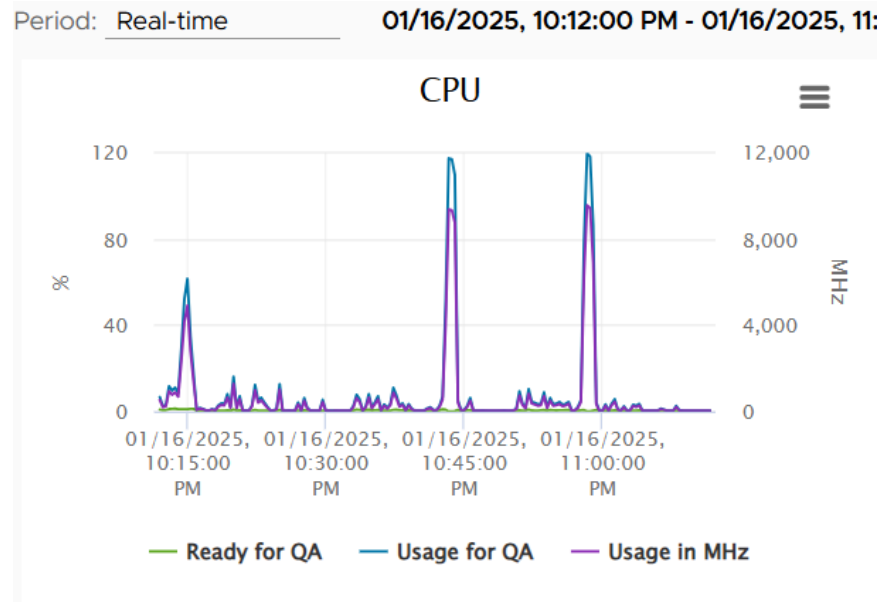
Моніторинг Ресурсів

Віртуальна машина, на якій проводяться тести, розгорнута на платформі VMware® vSphere, що дозволяє ефективно керувати ресурсами, такими як CPU,

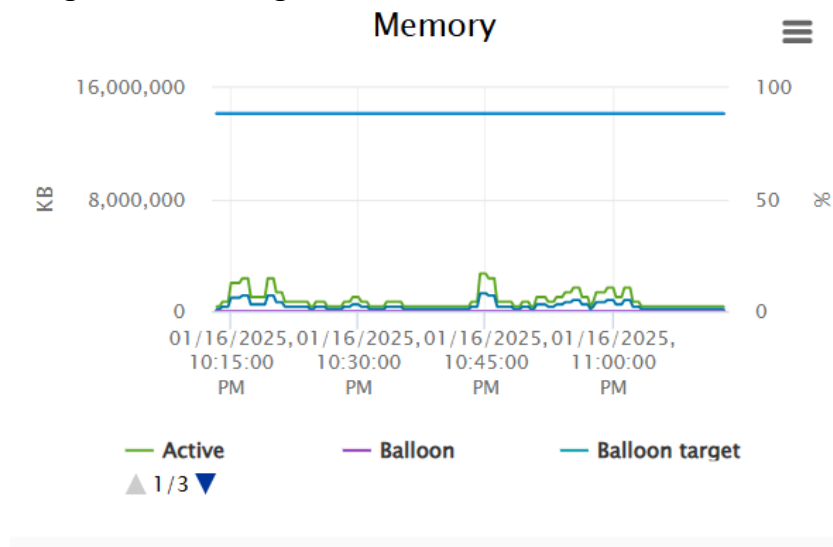
пам'ять, диск і мережа, для забезпечення стабільної роботи системи під навантаженням.

Моніторинг ресурсів віртуальної машини на платформі VMware® vSphere включає відстеження використання CPU, пам'яті, дисків і мережі для забезпечення стабільної роботи. Основні параметри:

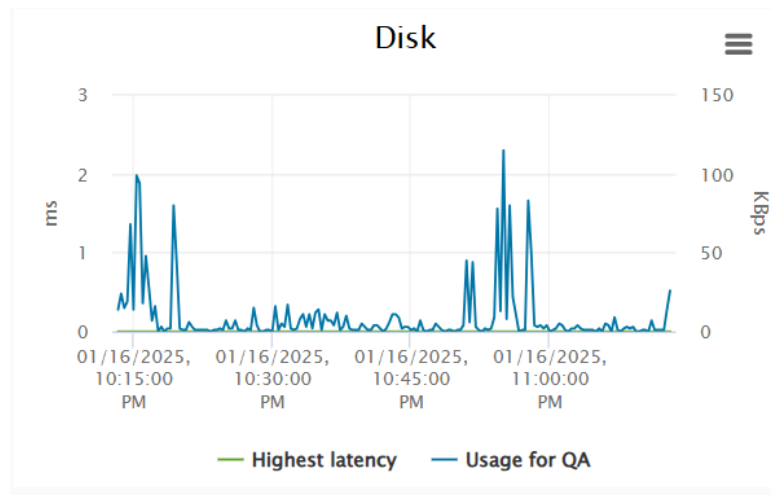
1. **CPU:** використання процесора, затримка доступу до ресурсів.



2. **Пам'ять:** використання оперативної пам'яті, механізм балонінгу.



3. **Диск:** використання диска, затримки при записі та читанні.



4. Мережа: завантаження мережі, затримка і помилки.

